

LLMRepair Pipeline 说明

基于 LLM 语义 Profiling 和 RNNRepair 式状态抽象的 Who&When 失败归因

Model-based LLM Agent Debugging

2026 年 6 月 28 日

1 我们要解决什么问题

Who&When 数据集里的每一条样本是一段失败的多智能体执行日志。日志里有一个任务问题、多个 agent 的历史对话或工具调用，以及人工标注的两个答案：

- **Who:** 哪个 agent 对最终失败负主要责任，即 `mistake_agent`。
- **When:** 决定性错误最早发生在哪一步，即 `mistake_step`。

我们的目标不是让 LLM 直接 all-at-once 输出“谁错了、哪一步错了”。我们的目标是做一个更结构化的方法：

LLM 先把每一步变成结构化语义标签；
再用类似 RNNRepair 的状态抽象方法，把这些标签变成 agent 状态机；
最后根据训练集中哪些状态更容易出错，定位新日志里的 Who 和 When。

所以这个方法仍然是 LLM-based，因为每一步的语义理解由 DeepSeek-v4-flash 完成；但最终定位不是一次性问 LLM，而是由一个可解释、可缓存、可交叉验证的状态模型完成。

2 当前已经跑通的实验配置

当前已经完成的是 **Hand-Crafted** 子集上的 profiling 和五折评估。
非 human step 的 agent 分布如下：

3 整体流程

整个 pipeline 可以分成六步：

1. 把 Who&When JSON 日志摊平成“每一步一行”的表。
2. 对每一个非 human step 调用 DeepSeek-v4-flash，得到结构化语义 profile。
3. 把语义 profile 转成数值特征。
4. 每个 agent 单独训练 GMM，把它的 step 聚成若干抽象状态。

项目	当前配置
数据集	Who&When Hand-Crafted
日志数量	58 条
总 step 数	2993
非 human step 数	2935
LLM	DeepSeek-v4-flash
是否给 ground truth answer	否, without_ground_truth
每步局部上下文窗口	当前 step 之前最多 4 步
评估方式	log-level 5-fold cross validation
状态模型	每个 agent 单独训练 PCA + GMM

Agent	step 数
Orchestrator	2283
WebSurfer	568
Assistant	39
FileSurfer	35
ComputerTerminal	10

5. 在训练集上统计每种状态或转移的历史错误风险，得到 risk table。
6. 在测试日志上逐步打分，预测 responsible agent 和 decisive error step。

一句话概括就是：

原始日志 → 每步语义 profile → agent 状态机 → 状态风险表 → Who/When 预测

4 第一步：把日志摊平成每步一行

原始 JSON 里，一条日志大致有这些内容：

- **question**: 任务问题。
- **history**: agent 交互历史，是一个 step 列表。
- **mistake_agent**: 人工标注的 responsible agent。
- **mistake_step**: 人工标注的 decisive error step。

我们先把每条日志的 **history** 摊平。摊平之后，每一行代表一个 step，核心字段如下：

字段	含义
log_id	属于哪一条日志。
step_idx	这是日志里的第几步，从 0 开始。
agent_name	归一化后的 agent 名字，例如 Orchestrator, WebSurfer。
raw_agent	原始 JSON 里的 agent 名字。
content	这一条 step 的实际文本或工具结果。
question	当前日志对应的任务问题。
ground_truth	如果数据里有最终正确答案，会保留；当前实验没有喂给 LLM。
mistake_agent	人工标注的错误 agent。
mistake_step	人工标注的错误 step。
is_decisive_error	如果 step_idx == mistake_step，则为 true。

这一步输出到：

artifacts/handcrafted_steps.jsonl

对应命令是：

```
python scripts/llmrepair_pipeline.py prepare-steps \
  --dataset-root datasets/Who_and_When \
  --output artifacts/handcrafted_steps.jsonl
```

5 第二步: LLM semantic profiling

这是整个方法里最关键的 LLM-based 部分。对每一个非 human step, 我们调用一次 DeepSeek-v4-flash。但是我们不~~不~~让它直接回答“谁错了、哪一步错了”。我们只让它判断当前 step 的局部语义状态。

5.1 每次 LLM 调用看什么

对于某个 step t , LLM 看到的是:

1. 当前日志的任务问题 `question`。
2. 当前 step 前面最多 4 步局部上下文。
3. 当前 step 的 `step_idx`。
4. 当前 step 的 agent 名字。
5. 当前 step 的内容 `content`。

当前实验是 `without_ground_truth`, 所以 prompt 里没有给最终正确答案。

注意, 这里不是把完整 history 都塞给 LLM。当前实现采用局部窗口:

$$\text{context}(t) = \{t-4, t-3, t-2, t-1\}$$

如果前面不足 4 步, 就给已有的前文。选择 4 不是理论常数, 而是当前工程默认值: 它能提供足够的局部因果上下文, 同时控制 token 成本和噪声。后续可以把窗口 0、2、4、8 做 ablation。

5.2 LLM 输出什么

LLM 必须返回一个 JSON 对象。当前字段是:

字段	含义
<code>action_type</code>	当前 step 的动作类型, 如 <code>plan</code> , <code>delegate</code> , <code>web_search</code> , <code>web_browse</code> , <code>tool_result</code> , <code>file_read</code> , <code>code_execute</code> , <code>reason</code> , <code>answer</code> , <code>termination</code> , <code>other</code> 。
<code>agent_intent</code>	agent 这一步想做什么, 短文本。
<code>task_alignment</code>	0 到 3 的整数, 越高表示越贴近任务。
<code>evidence_quality</code>	0 到 3 的整数, 越高表示证据越强。
<code>progress_state</code>	<code>on_track</code> , <code>uncertain</code> , <code>stalled</code> , <code>off_track</code> 之一。
<code>new_claim</code>	是否提出了新结论或新声明。
<code>unsupported_claim</code>	是否有缺少证据支持的声明。
<code>tool_use</code>	是否涉及工具使用。

字段	含义
<code>tool_error</code>	工具是否失败或返回异常。
<code>reasoning_error_signal</code>	是否出现明显推理错误信号。
<code>state_summary</code>	对当前局部状态的一句话总结。

可以把这一步理解成：LLM 是一个“语义传感器”。它不做最终判决，只把自然语言日志翻译成结构化观察。

5.3 为什么这样设计

直接 all-at-once 问 LLM 有几个问题：

- 长日志容易超上下文或注意力分散。
- LLM 的最终答案不稳定，输出格式也可能不稳定。
- 很难知道它到底为什么选某一步。

我们现在把任务拆开：

- LLM 负责它擅长的事：理解局部文本语义。
- 状态模型负责可重复的事：聚类、统计风险、交叉验证评估。

这使得结果更可缓存、更可解释，也更接近 RNNRepair 的思想。

6 第三步：把 profile 转成数值特征

LLM 输出的 JSON 不能直接喂给 GMM，所以要把它变成特征向量。

当前真正进入状态模型的字段是：

- 数值或布尔特征：`task_alignment`, `evidence_quality`, `new_claim`, `unsupported_claim`, `tool_use`, `tool_error`, `reasoning_error_signal`。
- 类别特征：`action_type`, `progress_state`。

类别特征用 one-hot 编码。例如 `action_type=web_search` 会变成一个二进制维度。

`agent_intent` 和 `state_summary` 当前主要用于人工解释，暂时没有进入 GMM。这样做是为了先保持 v1 简单、稳定。如果后面需要，可以把它们再 embedding 成向量加入特征。

7 第四步：每个 agent 单独建状态机

RNNRepair 的核心思想是：不要直接看原始高维 hidden state，而是先把 hidden state 聚成少数几个抽象状态，然后分析哪些状态和错误有关。

我们的对象不是 RNN hidden state，而是 LLM semantic profile。因此对应关系是：

RNNRepair	我们的方法	解释
RNN hidden state	LLM semantic feature	每一步的内部状态表示
GMM abstract state	GMM semantic state	把相似 step 聚成状态
state confidence/stability	semantic stability	判断状态语义是否纯
influence model	risk table	查某种状态历史上多危险

7.1 为什么每个 agent 单独建

不同 agent 的行为空间不一样。比如：

- WebSurfer 的状态可能是搜索、浏览、证据不足、网页错误。
- Orchestrator 的状态可能是规划、分派、总结、错误终止。
- ComputerTerminal 的状态可能是执行代码、工具输出、命令失败。

如果把所有 agent 混在一起聚类，状态会混乱。因此当前做法是：

for each agent a , fit a separate GMM on steps spoken by a

7.2 训练 GMM 前的处理

对每个 agent 的所有训练 step：

1. 把 profile 字段转成数值向量。
2. 用 `StandardScaler` 标准化。
3. 用 PCA 降维，当前最多降到 8 维。
4. 用 GMM 聚类得到抽象状态。

这里 PCA 的作用是让 GMM 更稳定。原始 one-hot 特征维度不算特别高，但数据量很小，尤其 Assistant、FileSurfer、ComputerTerminal 的样本很少，直接 GMM 容易不稳。

8 第五步：GMM 的 K 怎么选

这里我们参考 RNNRepair 的 “semantic-guided abstraction” 思路，不是只看 BIC，也不是用错误标签直接调 K 。

对候选 $K = 2, 3, \dots, 15$ ，我们都训练一个 GMM。然后看聚出来的每个状态语义是否稳定。

8.1 什么叫状态语义稳定

假设某个状态 q 里有很多 step。我们看这些 step 的语义字段是否一致。例如:

- 这个状态里的 `progress_state` 大多数是不是都是 `off_track`?
- 这个状态里的 `action_type` 大多数是不是都是 `web_browse`?
- 这个状态里的 `unsupported_claim` 大多数是不是都是 `true`?

对每个语义字段, 我们计算纯度:

$$\text{purity}(q, c) = \frac{\text{状态 } q \text{ 中字段 } c \text{ 的最大类别计数}}{\text{状态 } q \text{ 的样本数}}$$

然后对所有语义字段取平均, 得到这个状态的 `stability`:

$$\text{stability}(q) = \frac{1}{|C|} \sum_{c \in C} \text{purity}(q, c)$$

再对所有状态取平均:

$$\text{avg_stability}(K) = \frac{1}{K} \sum_{q=1}^K \text{stability}(q)$$

8.2 当前选择规则

当前规则是:

从小到大尝试 K , 选择第一个满足

$$\text{avg_stability}(K) \geq 0.72$$

并且每个状态至少有 5 个样本的 K 。如果没有 K 同时满足, 就选择 `stability` 最好的可用 K 。

这和 RNNRepair 的精神一致: 我们希望状态不是为了贴标签而过拟合, 而是语义上真的能解释。

在全量 hand-crafted profile 上, 当前选出来的 K 是:

Agent	Selected K	选择原因
Orchestrator	2	达到 <code>stability</code> 阈值
WebSurfer	3	达到 <code>stability</code> 阈值
Assistant	2	达到 <code>stability</code> 阈值
FileSurfer	2	达到 <code>stability</code> 阈值
ComputerTerminal	2	小样本下的 best available

9 第六步：训练 risk table

有了每一步的 agent 状态之后，我们要统计：训练集中哪些状态更容易成为决定性错误。
首先定义全局基础错误率：

$$p_0 = \frac{\text{训练集中决定性错误 step 数}}{\text{训练集中可用 step 数}}$$

然后对某个 key，例如某个 agent 的某个状态，计算：

$$p(\text{error} \mid \text{key}) = \frac{\text{key 下错误 step 数} + \alpha}{\text{key 下总 step 数} + 2\alpha}$$

这里使用 Laplace smoothing，当前：

$$\alpha = 0.5$$

最后定义：

$$\text{risk_lift}(\text{key}) = \frac{p(\text{error} \mid \text{key})}{p_0}$$

直观解释：

- `risk_lift = 1`: 这个状态和平均水平一样危险。
- `risk_lift > 1`: 这个状态比平均更容易出错。
- `risk_lift < 1`: 这个状态比平均更安全。

9.1 risk table 的 key

当前有四层 key，从具体到粗糙依次回退：

1. **transition**: (*agent*, *previous_state*, *action_type*, *current_state*)
2. **state**: (*agent*, *current_state*)
3. **action**: (*agent*, *action_type*)
4. **agent**: (*agent*)

预测时先查最具体的 transition。如果样本支持数不够，就回退到 state，再回退到 action，再回退到 agent。如果都没有足够支持，就使用基础风险，即 `risk_lift = 1`。

9.2 为什么训练时只用 `mistake_step` 之前的 step

训练 risk table 时，只使用：

$$\text{step_idx} \leq \text{mistake_step}$$

原因是：决定性错误之后的 step 往往是错误的后果，而不是错误的原因。如果把后续 step 也当作正常训练样本，模型可能会学到“错误之后的混乱状态”，而不是“错误发生的状态”。

10 第七步：每一步怎么打分

测试一条新日志时，我们对每个非 human step 都算三个东西：

1. `profile_score`: LLM 语义 profile 自身显示的局部可疑程度。
2. `risk_lift`: 这个状态在训练集中历史上有多危险。
3. `combined_score`: 把上面两者合起来的总分。

10.1 `profile_score`

当前公式是：

$$\begin{aligned} \text{profile_score} = & 2.0(3 - \text{task_alignment}) + 0.75(3 - \text{evidence_quality}) \\ & + \text{progress_penalty} + 2.0 \cdot I(\text{reasoning_error_signal}) \\ & + 1.5 \cdot I(\text{tool_error}) + 1.5 \cdot I(\text{unsupported_claim}) \end{aligned}$$

其中：

$$\text{progress_penalty} = \begin{cases} 0, & \text{on_track} \\ 0.25, & \text{uncertain} \\ 1.0, & \text{stalled} \\ 2.0, & \text{off_track} \end{cases}$$

这个分数只看 LLM 对当前 step 的语义判断。比如 task alignment 很低、evidence quality 很低、又有 unsupported claim，那么 profile score 就会高。

10.2 `combined_score`

当前组合公式是：

$$\text{combined_score} = \text{profile_score} + 2 \log(\text{risk_lift})$$

这里的含义是：

- `profile_score`: 当前 step 的文本内容本身像不像错误。
- `risk_lift`: 训练集中同类状态是否真的经常出错。

为什么用 log? 因为 risk lift 是倍数。如果直接加倍数，少数小样本高风险状态会把分数拉得过猛。取 log 后：

$$\log(1) = 0, \quad \log(2) \approx 0.69, \quad \log(4) \approx 1.39, \quad \log(0.5) \approx -0.69$$

所以：

- `risk_lift = 1` 时，不改变分数。
- `risk_lift > 1` 时，加分。
- `risk_lift < 1` 时，扣分。
- 特别大的 lift 会被 log 压住。

前面的 2 是当前经验权重，意思是让历史状态风险有明显影响，但不完全压过 LLM 的局部语义判断。后续论文实验里，这个权重应该做验证集调参和 ablation。

11 第八步：怎么从 step 分数得到 Who 和 When

当前实现了五种 decoding 模式：

模式	含义
<code>profile_argmax</code>	选择 <code>profile_score</code> 最高的 step。
<code>profile_earliest4</code>	选择最早一个 <code>profile_score</code> ≥ 4 的 step；如果没有，再退回 <code>argmax</code> 。
<code>risk_argmax</code>	选择 <code>risk_lift</code> 最高的 step。
<code>combined_argmax</code>	选择 <code>combined_score</code> 最高的 step。
<code>combined_earliest4</code>	选择最早一个 <code>combined_score</code> ≥ 4 的 step；如果没有，再退回 <code>combined argmax</code> 。

如果预测出来的 step 是第 t 步，那么：

- **When** 就预测为 t 。
- **Who** 就预测为第 t 步对应的 agent。

当前建议报告两个主结果：

- **LLMRepair-Agent**: 使用 `combined_argmax`，因为它的 agent-level accuracy 最高。
- **LLMRepair-Step**: 使用 `combined_earliest4`，因为它更符合“最早决定性错误”的定义，step 容差指标更好。

12 第九步：五折交叉验证怎么做

我们用 5-fold cross validation，并且按 log 切分，而不是按 step 切分。

每一折：

1. 58 条日志被分成训练日志和测试日志。

2. 只用训练日志拟合每个 agent 的 PCA + GMM。
3. 只用训练日志构建 risk table。
4. 在测试日志上预测 Who 和 When。
5. 收集 5 折所有测试预测，最后统一算指标。

这样可以避免同一条日志的一部分 step 进入训练、另一部分 step 进入测试造成泄漏。

LLM semantic profile 是 label-free 的，也就是 prompt 不包含 `mistake_agent` 或 `mistake_step`，因此可以预先缓存。真正使用人工错误标签的是 GMM 之后的 risk table 和评估阶段。

13 当前结果

当前 hand-crafted 58 条日志上的五折结果如下：

方法	Agent Acc.	Step Acc.	Tol@3	Tol@5	Avg Dist.
profile_argmax	56.90%	10.34%	27.59%	39.66%	17.79
profile_earliest4	53.45%	15.52%	34.48%	53.45%	9.16
risk_argmax	56.90%	22.41%	31.03%	46.55%	14.43
combined_argmax	70.69%	18.97%	31.03%	41.38%	17.95
combined_earliest4	58.62%	20.69%	39.66%	58.62%	9.29

这个表的解读是：

- 如果主要看 **Who**，combined_argmax 最强，agent accuracy 达到 70.69%。
- 如果主要看 **When**，risk_argmax 的 exact step accuracy 最高，为 22.41%。
- 但 combined_earliest4 的容差指标更好，Tol@5 达到 58.62%，也更符合“最早错误”的定义。
- profile_score 和 risk_lift 结合后，确实比只用其中之一更适合 agent attribution。

14 和论文 baseline 的关系

论文里的 all-at-once、step-by-step、binary-search 是直接让 LLM 判断 Who 和 When。我们这里的方法不同：

- 论文 baseline 是 **zero-shot LLM judge**。
- 我们的方法是 **LLM semantic profiler + trainable state/risk model**。

方法	Agent Acc.	Step Acc.
All-at-once	53.44%	3.51%
Step-by-step	32.75%	8.77%
Binary search	36.21%	6.90%

所以比较时必须说明：我们不是同一种 inference setting。我们的优势是可以从训练折学习状态风险；代价是需要标注训练日志。

根据本地整理的论文 hand-crafted 无 ground truth 数字，GPT-4o baseline 大致是：

从数值上看，我们当前结果在 hand-crafted 上明显高于这些 baseline，尤其是 agent-level 和 step exact。但论文写作时要避免夸大，必须写清楚这是 **supervised cross-validation setting**，不是纯 zero-shot。

15 实际运行命令

完整命令如下。这里不写 API key，运行前需要在环境变量中设置 DEEPSEEK_API_KEY。

1. 摊平 hand-crafted logs

```
python scripts/llmrepair_pipeline.py prepare-steps \
  --dataset-root datasets/Who_and_When \
  --output artifacts/handcrafted_steps.jsonl
```

2. 对每个非 human step 调 DeepSeek 做 semantic profiling

```
python scripts/llmrepair_pipeline.py profile-steps \
  --steps artifacts/handcrafted_steps.jsonl \
  --output artifacts/profiles/deepseek-v4-flash_without_gt.jsonl \
  --model deepseek-v4-flash \
  --ground-truth-mode without_ground_truth \
  --workers 4
```

3. 全量 profile 上看每个 agent 的 K 选择

```
python scripts/llmrepair_pipeline.py select-k \
  --profiles artifacts/profiles/deepseek-v4-flash_without_gt.jsonl \
  --output artifacts/state_models/k_selection_without_gt.json \
  --theta 0.72 \
  --min-support 5
```

4. 五折评估

```
python scripts/llmrepair_pipeline.py evaluate \
  --steps artifacts/handcrafted_steps.jsonl \
```

```
--profiles artifacts/profiles/deepseek-v4-flash_without_gt.jsonl \  
--output-dir artifacts/eval/without_gt \  
--n-splits 5 \  
--random-state 0
```

当前 profiling 已经完成，缓存文件里有 2935 个非 human step 的 profile。总 token 统计约为：

- prompt tokens: 2,675,846
- completion tokens: 1,279,194
- total tokens: 3,955,040

16 这个方法的主要创新点

当前 pipeline 的创新点可以这样概括：

1. **把 LLM 从最终裁判改成语义传感器。** LLM 不直接输出 Who/When，而是给每一步产生结构化语义状态。
2. **把 RNNRepair 的状态抽象迁移到 LLM agent 日志。** 原来 RNNRepair 聚类 RNN hidden state；我们聚类 LLM semantic profile。
3. **每个 agent 建自己的状态机。** 这比全局聚类更符合多智能体系统的角色差异。
4. **用状态风险表做可解释归因。** 每个预测都能回看：它是因为 profile 可疑，还是因为历史上同类状态风险高。
5. **支持缓存。** LLM profiling 可以缓存；后续改 GMM、risk table、decoding 策略时，不必重复调用 LLM。

17 目前还需要补强的地方

如果要往论文级别推进，建议下一步补这些实验：

1. **调参要更严格。** 当前 $\lambda = 2$ 和 earliest 阈值 4 是经验值，应该在 validation set 上调，或者做 nested CV。
2. **做 ablation。** 比较 only profile、only risk、combined、不同 window、不同 K 选择策略。
3. **补 algorithm-generated 子集。** 当前完整跑通的是 hand-crafted，需要看算法生成子集上是否也有效。
4. **补 DeepSeek baseline。** 本地目前只找到 DeepSeek-v4-flash all-at-once 结果，还缺 step-by-step 和 binary-search 的同模型复现。
5. **增强文本特征。** 当前 agent_intent 和 state_summary 没进模型，后续可以 embedding 后加入状态表示。

18 一句话版本

我们的方法是：先让 LLM 逐步阅读日志，把每一步翻译成“动作类型、证据质量、是否偏题、是否有 unsupported claim、工具是否错误”这类结构化语义标签；然后像 RNNRepair 一样，把同一个 agent 的语义标签聚成少数几个状态；再从训练日志里统计哪些状态和状态转移最容易对应决定性错误；最后在新日志中逐步打分，选择最可疑的 agent 和最早错误 step。